

BCSE204L Design and Analysis of Algorithms

Module-2

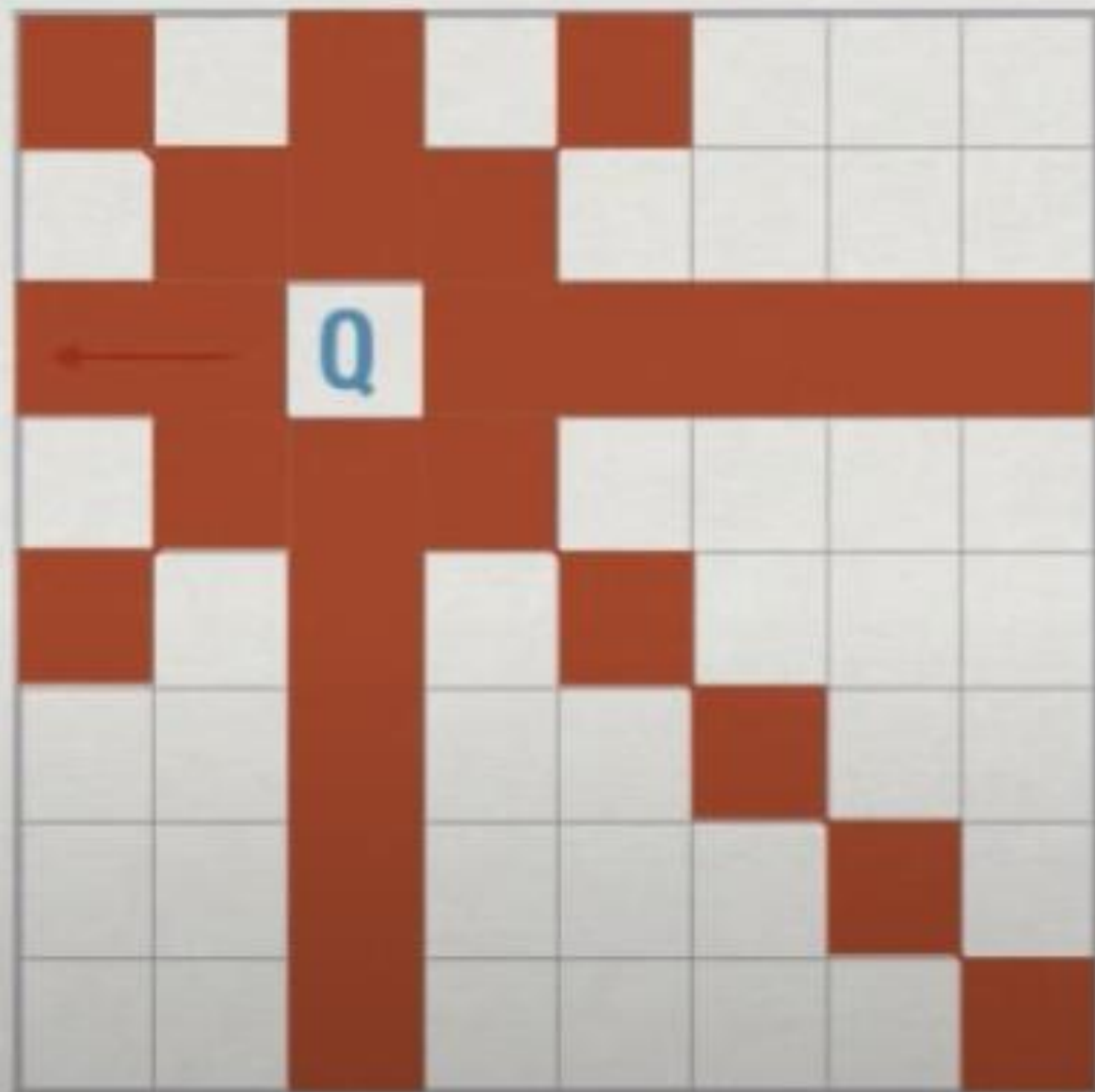
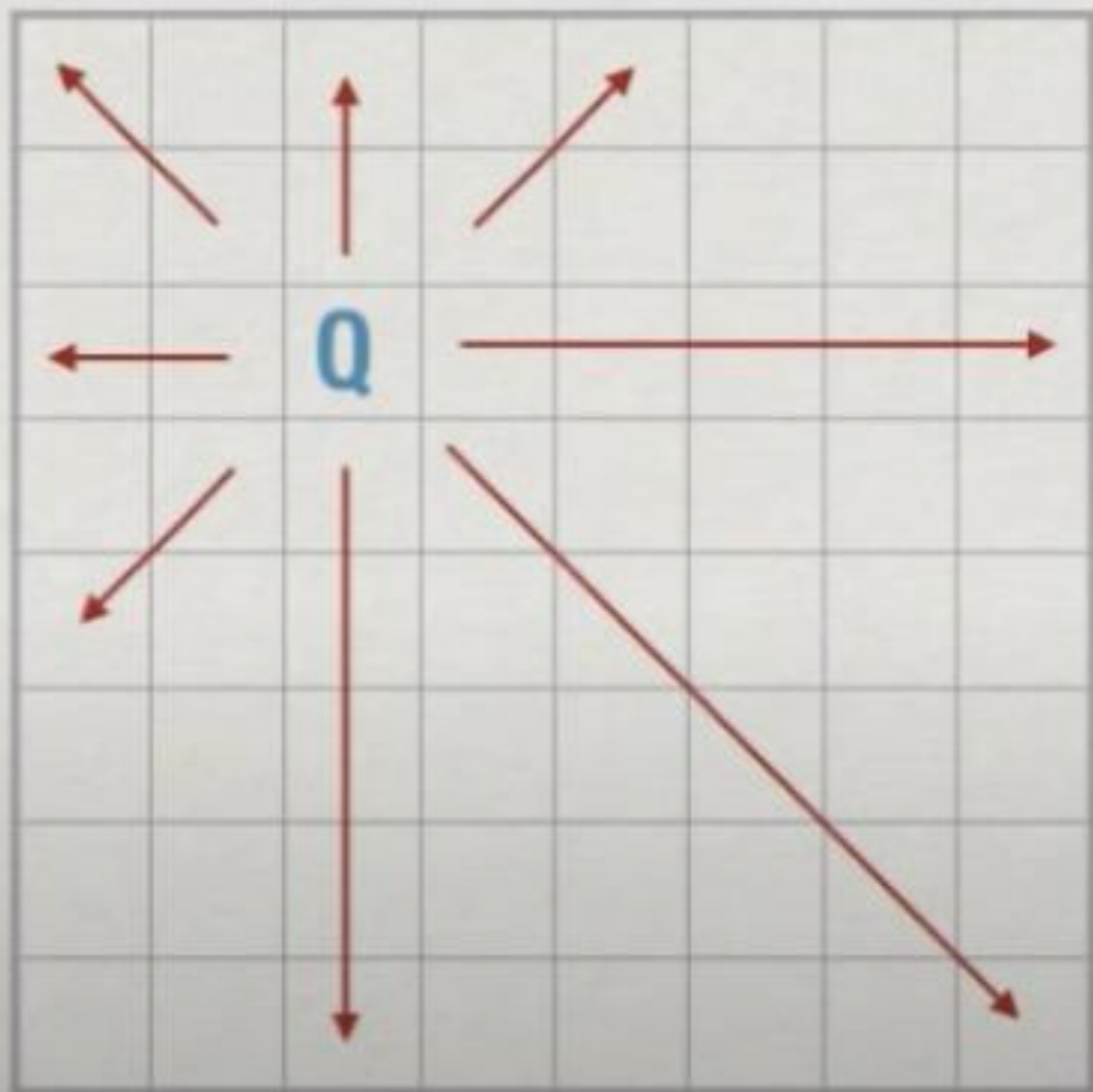
Topic-6: N Queens Problem

Backtracking- The concept

- Many solution, we need all
- Systematically search for the solution
- Build one step at a time
- If we hit a dead end (or leading to incorrect solutions)
 - Undo the last step
 - Try the next option

8 Queens Problem

- Place 8 queens on a chess board so that none of them attack each other
- In chess, a queen can move any number of squares along a row, column or diagonal
- 8 is clearly the limit



N Queens Problem

- Place N queens on an $N \times N$ chess board so that none attack each other
- $N = 1 \rightarrow$ trivial, $N = 2, 3 \rightarrow$ impossible
- $N = 4$ onward, solution exists

Solution Strategy

- Clearly exactly one queen in each row and column
- Place queens row by row
- In each row, place the queen in the first available (not under attack) column
- Keep trying to extend the next solution
- If we cannot, undo the previous move and try again (backtrack)
- Exhaustively search through all possibilities,
- but systematically

Demo

- On board

Logic

- Place the Q_1 in row_1 , Q_2 in row_2 etc.
- Then we need denote the column numbers only
- The solution would be a list (say X) of size N
- Eg :- Solutions to 4 Queens problem are $[2,4,1,3]$ and $[3,1,4,2]$

How to safely place queens?

- An individual square (i, j) is attacked by up to 4 queens
 - queen on row i and on column j
 - one queen on each diagonal through (i, j)
- Row i is overruled as we place one queen per row
- Column j can be checked as $X[j] == i$
- How to check diagonal attacks??

Checking diagonal attacks

- If two queens are placed at position (i, j) and (k, l)
- Then they are on same main diagonal (NW to SE) only if $(i - j) = (k - l)$ or $(j - l) = (i - k)$ and
- They are on same off diagonal (SW to NE) only if $(i + j) = (k + l)$ or $(j - l) = (k - i)$
- Combining both
- $|j - l| = |i - k|$

Pseudocode

- x is a global array (The answer)
- The code starts from $k = 1$
- Queen number and row number would be the same since we place 1st queen in the 1st row, 2nd queen in the 2nd row etc.
- $Place(k, i)$ function returns *True* if k^{th} queen can be placed in i^{th} column $\rightarrow$ Equivalently in cell (k, i) ; otherwise it returns *False*
- Recursion under iteration (Speciality of backtracking)

Place (k, i)

```
{  
  For j ← 1 to k - 1  
    do if (x [j] = i)  
      or (Abs x [j]) - i) = (Abs (j - k))  
    then return false;  
  return true;  
}
```

N - Queens (k, n)

```
{  
  For i ← 1 to n  
    do if Place (k, i) then  
    {  
      x [k] ← i;  
      if (k == n) then  
        write (x [1....n]);  
      else  
        N - Queens (k + 1, n);  
    }  
}
```

Time Complexity

- $T(n) = nT(n - 1) + O(n^2)$
- Because n recursive calls of $T(n - 1)$ and *Place* function, which is of $O(n)$ complexity would be called n times
- Solving the recurrence,
- $O(n!) = O(n^n)$

References

- Inspired by
- NPTEL Course on “Programming, Data Structures and Algorithms Using Python” by Prof. Madhavan Mukund, of Chennai Mathematical Institute (CMI)

Thank you...!!